

建置環境
作業系統：Win 10 IDE：Visual Studio Code 程式語言：Python 3
操作方式
1. 產生金鑰：python ./RSA.py init {size} 2. 加密：python ./RSA.py -e {plaintext} {n} {e} 3. 解密：python ./RSA.py -d {ciphertext} {n} {d}
執行結果
<pre> PS D:\Aya\台科\三上\資訊安全導論\HW\Information_Security_Class\hw4\B10632032_黃千綾> python ./RSA.py init 1024 p: 1113197469088712785256014087873347320288122524211508122763895843440000852266297276143329670385813158119444801123365 5610910574913969854907818312773011391436422339304643550645563506571478130691840985444823010470768319109772120514709101 6152341395136115726806435432974752404922785206242149042691534057825562054819 q: 1507530586822109606175634428325044596783989153394988753831129754605819915304571431356845675178797966252980209448129 3945564905065115654235790542335230102685639042070841933501456371328813583758718693512040285152249556140907483370030614 2381146545287722348090215529641500696877331670621252790461511357439850756111 n: 1678179233824194404050925725946229704223114725827055291376953739242679138226931691013926321988689278530665181481847 5819593808478923604236882647530523941451955286855846782380729805274494729268238833483251559465625909060389013445073886 4169354994732028845093075180716148967042970639185314006619060953839851811930207347603688599190131760717292539717275777 8162430493320014864097191991230351675588590433547894831230830115796255078561997039092924514668670280388678293575519387 8156329664676182255900077488740246469916610513408170198891132181813164399954133823508430544325355221142009897611726428 654382090306545738687981248909 e: 1677495422502154078724154825425737192975046987798700391139515330260963559190972025692217913711190101064575325025018 5137925134729143516978549893496361357503545066815531080127835267294368537599757224235030232335892242435133880464770875 2059971075878329100816337917038690229627585961830727807417083072096930424271 d: 1012413611572475134509229849835356483946246268958429734800282644104664199444742471272648622237388867830684422635689 4570925078219797352141839358775872332815355223920690981220838919185893042561960012607366369775589183966957728836024490 8387186734333260868440741919426597980035314207255766650096579313966872818272250541997835081816933587522886092703485334 4215209588509367447120296020017710608172498864201522939710499617826987259936782291585594827261639450729459528486009818 1072741082153273759003572398142023317194844791606502545799031117963243720430528659460220109933876599078076469683645446 87386823018021081289245660851 </pre>
<pre> PS D:\Aya\台科\三上\資訊安全導論\HW\Information_Security_Class\hw4\B10632032_黃千綾> python ./RSA.py -e abcdef 1678179233824194404050925 7259462297042231147258270552913769537392426791382269316910139263219886892785306651814818475819593808478923604236882647530523941451955286 8558467823807298052744947292682388334832515594656259090603890134450738864169354994732028845093075180716148967042970639185314006619060953 8398518119302073476036885991901317607172925397172757778162430493320014864097191991230351675588590433547894831230830115796255078561997039 0929245146686702803886782935755193878156329664676182255900077488740246469916610513408170198891132181813164399954133823508430544325355221 142009897611726428654382090306545738687981248909 167749542250215407872415482542573719297504698779870039113951533026096355919097202569221 7913711190101064575325025018513792513472914351697854989349636135750354506681553108012783526729436853759975722423503023233589224243513388 04647708752059971075878329100816337917038690229627585961830727807417083072096930424271 *jg#y! </pre>
<pre> PS D:\Aya\台科\三上\資訊安全導論\HW\Information_Security_Class\hw4\B10632032_黃千綾> python ./RSA.py -d *Jg#y! 1678179233824194404050925 7259462297042231147258270552913769537392426791382269316910139263219886892785306651814818475819593808478923604236882647530523941451955286 8558467823807298052744947292682388334832515594656259090603890134450738864169354994732028845093075180716148967042970639185314006619060953 8398518119302073476036885991901317607172925397172757778162430493320014864097191991230351675588590433547894831230830115796255078561997039 0929245146686702803886782935755193878156329664676182255900077488740246469916610513408170198891132181813164399954133823508430544325355221 142009897611726428654382090306545738687981248909 101241361157247513450922984983535648394624626895842973480028264410466419944474247127264 8622237388867830684422635689457092507821979735214183935877587233281535522392069098122083891918589304256196001260736636977558918396695772 8836024490838718673433326086844074191942659798003531420725576665009657931396687281827225054199783508181693358752288609270348533442152095 8850936744712029602001771060817249886420152293971049961782698725993678229158559482726163945072945952848600981810727410821532737590035723 9814202331719484479160650254579903111796324372043052865946022010993387659907807646968364544687386823018021081289245660851 9cj6#h </pre>
程式碼解說
main(): 讀取指令並印出結果

```
6 def main():
7     ...# ./RSA.py init {size}
8     ...if len(sys.argv) == 3:
9         ...if sys.argv[1] == 'init':
10            ...generate_key(int(sys.argv[2]))
11    ...elif len(sys.argv) == 5:
12        ...# ./RSA.py -e {plaintext} {n} {e}
13        ...if sys.argv[1] == '-e':
14            ...print(encrypt(sys.argv[2], int(sys.argv[3]), int(sys.argv[4])))
15        ...# ./RSA.py -d {plaintext} {n} {d}
16        ...elif sys.argv[1] == '-d':
17            ...print(decrypt(sys.argv[2], int(sys.argv[3]), int(sys.argv[4])))
18    ...# end main()
```

- generate_key(size)：產生兩個大質數及金鑰

```
21 def generate_key(size):
22     ...# 產生隨機質數 p 和 q
23     ...p = None
24     ...while p == None:
25         ...temp = random.randrange(2**(size-1), 2**size)
26         ...if is_prime(temp):
27             ...p = temp
28     ...print('p:', p)
29     ...q = None
30     ...while q == None:
31         ...temp = random.randrange(2**(size-1), 2**size)
32         ...if is_prime(temp) and temp != p:
33             ...q = temp
34     ...print('q:', q)
35     ...n = p*q
36     ...print('n:', n)
```

```
37     ... phiN = (p-1)*(q-1)
38     ... # 從 1 -- phiN 中取一個與 phiN 互質的 e
39     ... e = None
40     ... while(e == None):
41     ...     ... temp = random.randrange(2**(size-1), 2**size)
42     ...     ... if(math.gcd(temp, phiN) == 1):
43     ...         ... e = temp
44     ...         ... break
45     ... print('e:', e)
46     ... # 找一個 d 使 d * e ≡ 1 mod phiN
47     ... d = find_mode_inverse(e, phiN)
48     ... print('d:', d)
49     ... return p, q, n, e, d
50 # end generate_key()
```

- is_prime(number)：判斷是否為質數（小質數使用一般判斷方式，大質數用 rabin_miller）

```
77 def is_prime(number): ... # 判斷質數
78
79     ... if(number < 2):
80     ...     ... return False
81     ... elif(number == 2):
82     ...     ... return True
83     ... elif(number % 2 == 0):
84     ...     ... return False
85     ... # 測試 2 -- sqrt(n) 中有無 n 的因數
86     ... elif(number < 1000):
87     ...     ... temp = math.ceil(math.sqrt(number))
88     ...     ... for i in range(3, temp, 2):
89     ...         ... if(number % i == 0):
90     ...             ... return False
91     ...     ... return True
92     ... else:
93     ...     ... return rabin_miller(number)
94 # end is_prime()
```

- rabin_miller(number)：測試是否為質數

```

97 def rabin_miller(number):
98     ... # 找 u 和 r 使  $p'-1 = 2^{**u} * r$ 
99     ... r = number - 1
100    ... u = 0
101    ... while r % 2 == 0:
102    ...     ... r = r // 2
103    ...     ... u += 1

```

```

104    ... # 做五次測試
105    ... for test in range(5):
106    ...     ... # 找 a 使  $a^{**r} \% p' \neq 1$ 
107    ...     ... a = random.randrange(2, number - 1)
108    ...     ... temp = square_and_multiply(a, r, number)
109    ...     ... if temp != 1:
110    ...     ...     ... # 找 a 使  $a^{**r^{**2j}} \neq (p'-1) \% p'$ 
111    ...     ...     ... j = 0
112    ...     ...     ... while temp != (number - 1):
113    ...     ...     ...     ... if j == u - 1:
114    ...     ...     ...         ... # 找到的話為合數
115    ...     ...     ...         ... return False
116    ...     ...     ... else:
117    ...     ...     ...         ... j = j + 1
118    ...     ...     ...         ... temp = square_and_multiply(temp, 2, number)
119    ...     ... # 找不到就有可能是質數

```

- square_and_multiply(x, h, n) : 加速計算 $x^{**h} \% n$

```

53 def square_and_multiply(x, h, n): ... # Output  $x^{**h} \% n$ 
54     ... y = x
55     ... bitH = [int(x) for x in bin(h)[2:]]
56     ... for i in range(1, len(bitH)):
57     ...     ... y = y**2 % n
58     ...     ... if bitH[i] == 1:
59     ...     ...     ... y = y * x % n
60     ... return y
61     # end square_and_multiply

```

- encrypt(plaintext, n, e) : 將明文轉成對應的數字，加密後再轉回字元

```
124 def encrypt(plaintext, n, e):
125     ... y = 0
126     ... for i in range(len(plaintext)):
127         ... y += (ord(plaintext[i]) - 33) * 90**(len(plaintext) - i)
128     ... y = square_and_multiply(y, e, n)
129     ... ciphertext = ''
130     ... for i in range(len(plaintext)):
131         ... char = y % 90
132         ... y //= 90
133         ... char = chr(char + 33)
134         ... ciphertext = ciphertext + char
135     ... return ciphertext
136     ... # end encrypt()
```

- decrypt(ciphertext, n, d)：將密文轉成對應的數字，解密後再轉回字元

```
139 def decrypt(ciphertext, n, d):
140     ... x = 0
141     ... for i in range(len(ciphertext)):
142         ... x += (ord(ciphertext[i]) - 33) * 90**(len(ciphertext) - i)
143     ... x = square_and_multiply(x, d, n)
144     ... plaintext = ''
145     ... for i in range(len(ciphertext)):
146         ... char = x % 90
147         ... x //= 90
148         ... char = chr(char + 33)
149         ... plaintext = plaintext + char
150     ... return plaintext
151     ... # end decrypt()
```

遇到困難與心得

一開始我一直沒有弄懂要加密的對象是什麼，所以爬了很多文章，最後才忽然理解要把整個字串轉成數字。但是到了這一步卻發現解密的結果是錯誤的，找不到錯誤出在哪裡。不過在某些部分還是有成功的，像是加上 square_and_multiply 和 rabin_miller 之後的速度真的很快，不然原本的方式大概都要等好幾分鐘。